AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Damian Basta

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Wstęp i cel projektu **3**

2. Analiza wymagań i założenia projektu **3**

2.1. Wymagania funkcjonalne 3

2.2. Wymagania niefunkcjonalne i środowisko 4

3. Projektowanie systemu, architektura oraz kontrola wersji **4**

3.1. Diagram klas i relacje 5

3.2. Opis komponentów systemu 5

3.3. Wykorzystanie systemu kontroli wersji i platformy GitHub 6

3.3.1. Główny widok aplikacji GitHub Desktop 6

3.3.2. Zatwierdzanie zmian (Commity) i historia projektu 6

3.3.3. Zdalne repozytorium na platformie GitHub 8

4. Implementacja i omówienie plików nagłówkowych (.h) **8**

4.1. Moduł węzła listy (`Node.h`) 8

4.2. Moduł wzorca Fabryki (`NodeFactory.h`) 10

4.3. Moduł zarządzania listą (`DoublyLinkedList.h`) 11

5. Wnioski i podsumowanie **12**

5.1. Podsumowanie osiągniętych celów 12

Literatura **13**

Spis rysunków **13**

Spis tabel **14**

Spis listingów **15**

1. Wstęp i cel projektu

Celem niniejszego projektu jest zaprojektowanie, implementacja oraz dokumentacja struktury danych typu **lista dwukierunkowa** w języku C++. W projekcie zastosowano nowoczesne podejście do inżynierii oprogramowania, w tym wzorzec projektowy *Fabryka* (*Factory Method* za pośrednictwem klasy `NodeFactory`) do kontrolowanego tworzenia węzłów, system kontroli wersji **Git** wraz z repozytorium zdalnym na platformie **GitHub**, a także zautomatyzowane generowanie dokumentacji techniczną przy użyciu narzędzia **Doxygen** oraz kompilację do formatu PDF.

Listy dwukierunkowe należą dofundamentalnych dynamicznych struktur danych. W przeciwieństwie do tablic, pozwalają na efektywne dodawanie i usuwanie elementów w dowolnym miejscu struktury bez konieczności relokacji całego bloku pamięci, a dzięki dwukierunkowym wskaźnikom umożliwiają przechodzenie sekwencyjne zarówno w przód, jak i w tył.

W ramach pracy zrealizowano następujące wymagania funkcjonalne i niefunkcjonalne:

- Implementacja bezpiecznej alokacji pamięci przy użyciu wzorca projektowego Fabryki.
- Pełny zestaw operacji modyfikacji i przeglądania listy (dodawanie na początek/koniec/indeks, usuwanie, wyświetlanie).
- Zarządzanie kodem źródłowym przy użyciu systemu Git i platformy GitHub.
- Automatyczne generowanie dokumentacji kodu źródłowego za pomocą Doxygena.

2. Analiza wymagań i założenia projektu

Rozdział ten przedstawia szczegółową specyfikację wymagań funkcjonalnych oraz technologicznych stawianych przed implementowaną strukturą danych.

2.1. Wymagania funkcjonalne

Program realizuje kompletny zestaw operacji na liście dwukierunkowej, co zostało przetestowane w głównej funkcji aplikacji (`main.cpp`):

1. Dodanie elementu na początek listy.
2. Dodanie elementu na koniec listy.
3. Dodanie elementu pod wskazany indeks.
4. Usunięcie elementu z początku listy.
5. Usunięcie elementu z końca listy.
6. Usunięcie elementu spod wskazanego indeksu.
7. Wyświetlenie całej listy wprzód.
8. Wyświetlenie listy w odwrotnej kolejności (w tył).
9. Wyświetlenie następnego elementu względem wybranego węzła.
10. Wyświetlenie poprzedniego elementu względem wybranego węzła.
11. Całkowite czyszczenie listy (zwolnienie pamięci).

2.2. Wymagania niefunkcjonalne i środowisko

Opracowane oprogramowanie spełnia standardy języka C++ (C++11 lub nowszy). Założono modularną budowę kodu podzieloną na niezależne pliki nagłówkowe (`.h`) oraz pliki implementacyjne (`.cpp`):

- **Język programowania:** C++
- **Kompilator:** GCC / MinGW (`g++`)
- **Kontrola wersji:** Git / GitHub
- **Dokumentacja:** Doxygen z komentarzami w standardzie JavaDoc/Doxygen (`///`, `/** ... */`)

3. Projektowanie systemu, architektura oraz kontrola wersji

Rozdział ten prezentuje architekturę systemową projektu, strukturę powiązań między klasami, a także opisuje środowisko narzędziowe związane z kontrolą wersji przy użyciu systemów Git i GitHub.

3.1. Diagram klas i relacje

Architektura projektu opiera się na paradygmacie obiektowym. Kod źródłowy podzielono na modularne komponenty. Wzorzec projektowy *Fabryka* (*Factory Method*) oddziela proces tworzenia obiektów od ich reprezentacji. Klasa `NodeFactory` odpowiada za kontrolowane instancjonowanie obiektów klasy `Node` na stercie, natomiast klasa `DoublyLinkedList` zarządza logiką powiązań wskaźnikowych `head` oraz `tail`.

Schemat architektury modułów: Node ← NodeFactory ← DoublyLinkedList

Rys. 3.1. Uproszczony schemat powiązań między klasami projektu

3.2. Opis komponentów systemu

W skład systemu wchodzą następujące główne komponenty programistyczne:

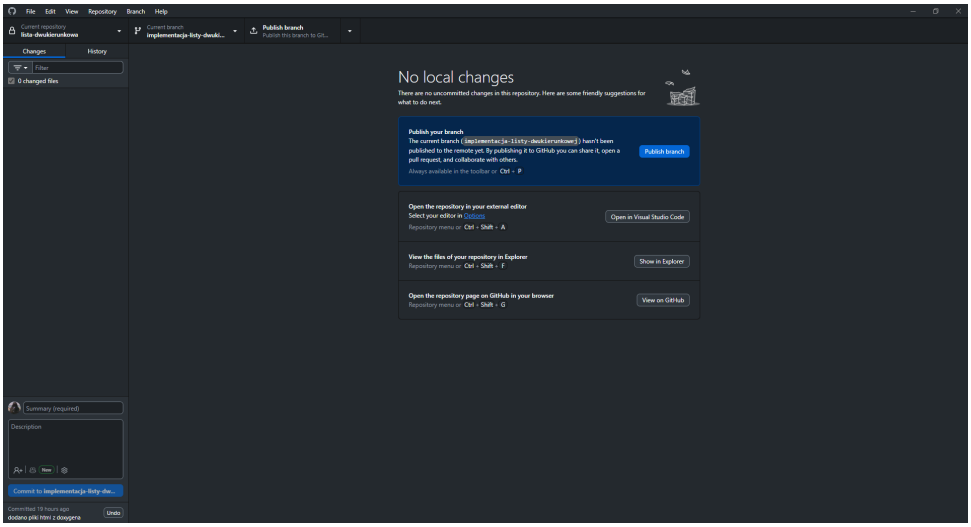
- `Node`: Klasa reprezentująca pojedynczy węzeł listy. Zawiera pole danych całkowitych (`data`) oraz wskaźniki `next` i `prev`.
- `NodeFactory`: Klasa implementująca statyczną metodę `createNode(int val)` służącą do bezpiecznego tworzenia nowych węzłów.
- `DoublyLinkedList`: Klasa zarządzająca kolekcją danych, implementująca operacje wstawiania, usuwania oraz przechodzenia przez strukturę w obu kierunkach.

3.3. Wykorzystanie systemu kontroli wersji i platformy GitHub

W procesie tworzenia oprogramowania kluczową rolę odgrywa kontrola wersji oraz praca z repozytorium zdalnym. Do zarządzania kodem źródłowym wykorzystano system **Git** oraz platformę **GitHub**, posiłkując się aplikacją graficzną **GitHub Desktop** w celu ułatwienia operacji commitowania oraz synchronizacji zmian.

3.3.1. Główny widok aplikacji GitHub Desktop

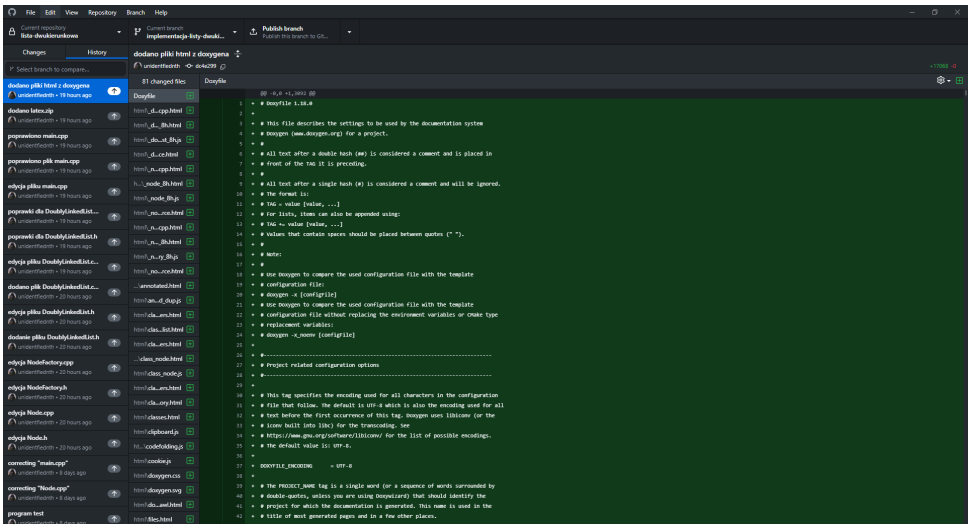
GitHub Desktop umożliwia intuicyjne śledzenie wprowadzonych w kodzie zmian, tworzenie gałęzi (*branches*) oraz zatwierdzanie modyfikacji bez konieczności korzystania z linii komend.



Rys. 3.2. Główny interfejs aplikacji GitHub Desktop podczas pracy nad projektem

3.3.2. Zatwierdzanie zmian (Commity) i historia projektu

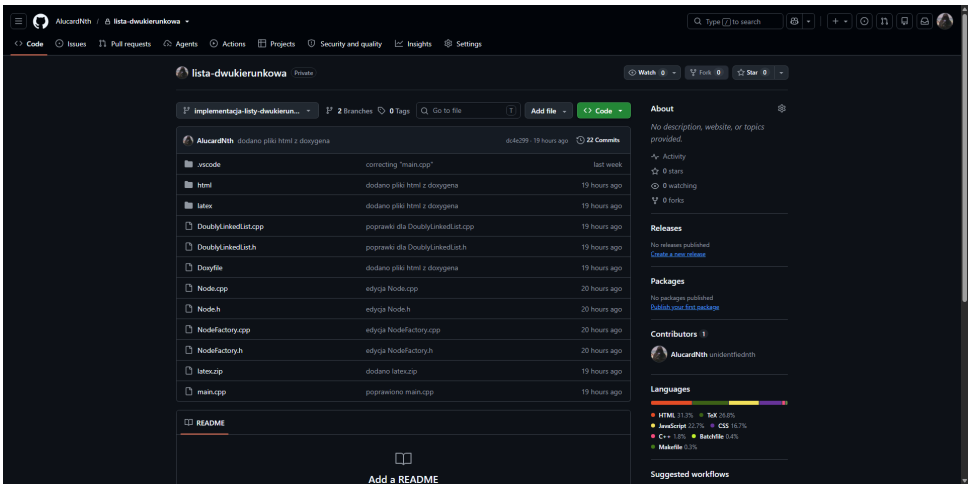
Kluczowym elementem kontroli wersji jest wykonywanie regularnych zatwierdzeń (*commitów*) z czytelnymi opisami, co pozwala kontrolować historię zmian i przywracać wcześniejsze wersje kodu.



Rys. 3.3. Lista zatwierdzeń (commitów) odzwierciedlająca postępy prac nad projektem

3.3.3. Zdalne repozytorium na platformie GitHub

Zsynchronizowany kod źródłowy został opublikowany w zdalnym repozytorium na platformie GitHub, co umożliwia podgląd struktury projektu z poziomu przeglądarki internetowej.



Rys. 3.4. Wygląd zdalnego repozytorium projektu w serwisie GitHub

4. Implementacja i omówienie plików nagłówkowych (.h)

W tym rozdziale przedstawiono oraz omówiono kompletne kody źródłowe wszystkich plików nagłówkowych (`.h`) wchodzących w skład modularnej architektury projektu. Pliki te definiują interfejsy klas, struktury danych oraz sygnatury metod udostępnianych poszczególnym modułom systemu.

4.1. Moduł węzła listy (`Node.h`)

Plik nagłówkowy `Node.h` odpowiada za definicję klasy reprezentującej pojedynczy element listy dwukierunkowej. Każdy węzeł przechowuje wartość typu całkowitego oraz dwa wskaźniki: na element następny oraz poprzedni.

```
1 #ifndef NODE_H
2 #define NODE_H
3
4 /**
5  * @file Node.h
6  * @brief Definicja klasy Node reprezentujacej wezel listy dwukierunkowej.
7  */
```

```
8  class Node {
9  public:
10     int data;      ///< Wartosc przechowywana w wezle
11     Node* next;    ///< Wskaznik na nastepny wezel w liscie
12     Node* prev;    ///< Wskaznik na poprzedni wezel w liscie
13
14     /**
15      * @brief Konstruktor inicjalizujacy wezel podana wartoscia.
16      * @param val Liczba calkowita zapisywana w wezle.
17      */
18     Node(int val);
19 };
20
21 #endif // NODE_H
```

Listing 1. Plik nagłówkowy węzła (Node.h)

Omówienie: Klasa `Node` stanowi fundament struktury. Zastosowanie modyfikatora publicznego dla pól danych i wskaźników ułatwia wewnętrzną manipulację strukturą w klasie zarządzającej, przy zachowaniu prostoty modelu danych.

4.2. Moduł wzorca Fabryki (`NodeFactory.h`)

Klasa `NodeFactory` realizuje wzorzec projektowy *Factory Method* (Fabryka), oddzielając logikę tworzenia obiektów węzłów od logiki zarządzania samą listą.

```
#ifndef NODE_FACTORY_H
#define NODE_FACTORY_H

#include "Node.h"

/**
 * @file NodeFactory.h
 * @brief Definicja klasy NodeFactory implementujacej wzorzec projektowy Factory.
 */
class NodeFactory {
public:
    /**
     * @brief Statyczna metoda tworzaca nowy wezel na stercie.
     * @param val Wartosc dla nowego wezla.
     * @return Wskaznik do utworzonego obiektu Node.
     */
    static Node* createNode(int val);
};

#endif // NODE_FACTORY_H
```

Listing 2. Plik nagłówkowy fabryki węzłów (NodeFactory.h)

Omówienie: Statyczna metoda `createNode` enkapsuluje operator `new`, co pozwala w przyszłości na łatwą rozbudowę mechanizmu alokacji pamięci (np. o pulę pamięci) bez konieczności modyfikacji kodu zarządzającego listą.

4.3. Moduł zarządzania listą (`DoublyLinkedList.h`)

Główny interfejs sterujący całą strukturą danych. Klasa `DoublyLinkedList` deklaruje wskaźniki krańcowe `head` oraz `tail` oraz udostępnia pełny zestaw operacji modyfikacji i traversingu.

```
#ifndef DOUBLY_LINKED_LIST_H
#define DOUBLY_LINKED_LIST_H

#include "Node.h"

/**
 * @file DoublyLinkedList.h
 * @brief Definicja klasy zarzadzajacej lista dwukierunkowa.
 */
class DoublyLinkedList {
private:
    Node* head; ///< Wskaznik na pierwszy element listy
    Node* tail; ///< Wskaznik na ostatni element listy

public:
    DoublyLinkedList();
    ~DoublyLinkedList();

    void prepend(int val);                  // Dodaj na poczatek listy
    void append(int val);                   // Dodaj na koniec listy
    void insertAtIndex(int index, int val); // Dodaj pod wskazany indeks
    void removeHead();                      // Usun element z poczatku
    void removeTail();                      // Usun element z konca
    void removeAtIndex(int index);          // Usun element spod indeksu
    void displayForward();                  // Wyswietl liste w przod
    void displayBackward();                 // Wyswietl liste w tyl
    void displayNext(Node* node);           // Wyswietl nastepny element
    void displayPrev(Node* node);           // Wyswietl poprzedni element
    void clear();                           // Zwolnij cala pamiec listy

    Node* getNode(int index);               // Pobierz wezel spod indeksu
};

#endif // DOUBLY_LINKED_LIST_H
```

Listing 3. Plik nagłówkowy listy dwukierunkowej (DoublyLinkedList.h)

Omówienie: Klasa ta integruje komponenty w spójną całość. Prywatne wskaźniki `head` i `tail` gwarantują hermetyzację stanu wewnętrznego, natomiast bogaty zestaw metod publicznych pozwala na elastyczne zarządzanie elementami struktury.

5. Wnioski i podsumowanie

Zrealizowany projekt pozwolił na utrwalenie wiedzy z zakresu zaawansowanego programowania obiektowego w języku C++, dynamicznej alokacji pamięci oraz operacji na wskaźnikach.

5.1. Podsumowanie osiągniętych celów

1. Poprawnie zaimplementowano listę dwukierunkową obsługującą pełny cykl życia elementów (dodawanie, usuwanie, przeglądanie w obu kierunkach).
2. Zastosowano wzorzec projektowy Fabryki (`NodeFactory`), co zwiększyło czytelność i modularność kodu.
3. Wykorzystano system kontroli wersji Git oraz platformę GitHub do zarządzania repozytorium kodu.
4. Wygenerowano profesjonalną dokumentację techniczną przy użyciu narzędzia Doxygen, co ułatwia dalszy rozwój i utrzymanie oprogramowania.

Spis rysunków

3.1. Uproszczony schemat powiązań między klasami projektu 5
3.2. Główny interfejs aplikacji GitHub Desktop podczas pracy nad projektem 6
3.3. Lista zatwierdzeń (commitów) odzwierciedlająca postępy prac nad projektem 7
3.4. Wygląd zdalnego repozytorium projektu w serwisie GitHub 8

Spis tabel

Spis listingów

1. Plik nagłówkowy węzła (Node.h) 8
2. Plik nagłówkowy fabryki węzłów (NodeFactory.h) 10
3. Plik nagłówkowy listy dwukierunkowej (DoublyLinkedList.h) 11